LAB S11. Dos bases más y una caché

Javier Velasco Pacheco
Grupo 2
Sistemas Distribuidos 2027-1

PARTE 0. Levantar y ver el punto de partida

Se realizaron consultas a las cuentas de Ana, Beto y Carla para comprobar que sus saldos fueran los establecidos al inicio de la práctica. Esta verificación permite contar con un punto de referencia antes de realizar las pruebas de transferencia y almacenamiento en caché.

```
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T db sh -c 'psql -U "$POSTGRES_USER" -d cuenta -c "SELECT clabe, titular, saldo, version FROM cuenta
WHERE clabe IN ('\''0021800000000000001'\'','\''0021800000000000002'\'','\''0021800000000000003'\'') ORDER BY clabe;"'
       clabe        | titular |  saldo  | version
--------------------+---------+---------+---------
 0021800000000000001 | Ana     | 1000.00 |       0
 0021800000000000002 | Beto    |  500.00 |       0
 0021800000000000003 | Carla   |  300.00 |       0
(3 rows)
```

Figura. Consultas de saldos iniciales

Se realizó una transferencia de $200 de la cuenta de Ana a la cuenta de Beto. Posteriormente, se consultaron los saldos de ambas cuentas para verificar que el cargo y el abono se aplicaran correctamente. De esta manera, se comprobó que la información se mantuviera actualizada después de una operación bancaria.

```
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/cuentas/0021800000000000001; echo
{"clabe":"0021800000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T00:43:59.224944008Z"}

Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/cuentas/0021800000000000002; echo
{"clabe":"0021800000000000002","titular":"Beto","saldo":700.00,"leidaEn":"2026-11-06T00:44:05.526455102Z"}
```

Figura. Consultas de saldos tras la transferencia

Se realizaron diez consultas consecutivas al saldo de Ana mediante el servicio de cuentas. El objetivo fue observar el comportamiento de la aplicación al solicitar repetidamente la misma información y comprobar si Redis permite reutilizar los datos almacenados en caché, evitando consultas innecesarias a PostgreSQL.

```
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 10); do
  curl -s http://localhost:8080/api/cuentas/0021800000000000001; echo
done
{"clabe":"0021800000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T00:42:53.016332113Z"}
{"clabe":"0021800000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T00:42:53.248893571Z"}
{"clabe":"0021800000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T00:42:53.470535120Z"}
{"clabe":"0021800000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T00:42:53.798907406Z"}
{"clabe":"0021800000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T00:42:54.155905157Z"}
{"clabe":"0021800000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T00:42:54.450038126Z"}
{"clabe":"0021800000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T00:42:54.687256157Z"}
{"clabe":"0021800000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T00:42:54.923669752Z"}
{"clabe":"0021800000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T00:42:55.126653665Z"}
{"clabe":"0021800000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T00:42:55.365807018Z"}
```

Figura. Consultas de saldo de Ana diez veces seguidas

Se revisaron los registros del servicio de cuentas para identificar las lecturas realizadas directamente desde PostgreSQL. Estos registros permiten observar cuántas veces la aplicación tuvo que consultar la base de datos para recuperar el saldo de Ana y evaluar si el uso de Redis reduce la carga de consultas.

```
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s11 (trabajo-s11)
$ docker compose logs cuenta | grep -c "leida de Postgres"
17
```

Figura. Cuentas de Ana en Log de postgres

Se consultaron las estadísticas de Redis para identificar los hits y misses de la caché. Los hits representan las solicitudes en las que la información se encontró en Redis, mientras que los misses indican que el dato solicitado no estaba disponible y fue necesario buscarlo en otra fuente, como PostgreSQL.

```
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli INFO stats | grep keyspace
keyspace_hits:0
keyspace_misses:0
```

Figura. Lecturas de fallo y hits

PARTE 1. La caché del saldo en Redis

- Enciende y configura la caché

El archivo ConfiguracionCache.java habilita el mecanismo de caché de Spring. Spring Boot ya crea el RedisCacheManager porque el proyecto incluye spring-boot-starter-data-redis; aquí se configura la caché cuentas. entryTtl establece el tiempo de vida, Jackson2JsonRedisSerializer guarda los objetos como JSON y transactionAware hace que el borrado espere a que termine la transacción.

```
package mx.mexibanco.cuenta;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.beans.factory.annotation.Value;
import
org.springframework.boot.autoconfigure.cache.RedisCacheManagerBuilderCustomizer;
import org.springframework.cache.annotation.EnableCaching;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.cache.RedisCacheConfiguration;
import org.springframework.data.redis.serializer.Jackson2JsonRedisSerializer;
import
org.springframework.data.redis.serializer.RedisSerializationContext.Serializatio
nPair;
import java.time.Duration;

@EnableCaching
@Configuration
```

```
public class ConfiguracionCache {
    public static final String CUENTAS = "cuentas";

    @Bean
    RedisCacheManagerBuilderCustomizer
cacheDeCuentas(@Value("${mexibanco.cache.ttl:30s}") Duration ttl, ObjectMapper
json) {
        RedisCacheConfiguration cuentas =
RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(ttl)
            .disableCachingNullValues()
            .serializeValuesWith(SerializationPair.fromSerializer(
                new Jackson2JsonRedisSerializer<>(json, CuentaVista.class)));
        return builder -> builder.withCacheConfiguration(CUENTAS, cuentas)
            .transactionAware(); // DEL espera al commit exitoso de la
transacción
    }
}
```

- Cachea la consulta que muestra

ConsultaDeSaldo.java es el método que atiende GET /cuentas/{clabe}. La anotación @Cacheable indica que Spring debe buscar primero el resultado en la caché cuentas usando #clabe como clave. Si hay un hit, el método no se ejecuta; si hay un miss, consulta PostgreSQL y el resultado se almacena.

```
package mx.mexibanco.cuenta;
import mx.mexibanco.compartido.NoEncontradoException;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.stereotype.Service;
@Service
public class ConsultaDeSaldo {

    private static final Logger log =
LoggerFactory.getLogger(ConsultaDeSaldo.class);
    private final CuentaRepository cuentas;

    public ConsultaDeSaldo(CuentaRepository cuentas) {
        this.cuentas = cuentas;
    }

    @Cacheable(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
    public CuentaVista consultar(String clabe) {
        // Esta linea sale en el log SOLO cuando se lee Postgres. Con la cache
puesta, en un hit no aparece.
        log.info("[consulta] {} leida de Postgres", clabe);
        return cuentas.findByClabe(clabe)
            .map(CuentaVista::de)
            .orElseThrow(() -> new NoEncontradoException("No existe la CLABE " +
clabe));}}
```

- Borra la copia cuando cambia el saldo

Los métodos cargar y abonar modifican el saldo. @CacheEvict elimina la entrada de la cuenta modificada. El borrado no sustituye la actualización de PostgreSQL: primero se confirma la transacción y después se elimina la copia temporal. La siguiente consulta vuelve a cargar el saldo actualizado.

```
import org.springframework.cache.annotation.CacheEvict; ...
@CacheEvict(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")

@Transactional public SaldoAplicado cargar(String clabe, BigDecimal monto,
String clave)
{ ... }
@CacheEvict(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
@Transactional public SaldoAplicado abonar(String clabe, BigDecimal monto,
String clave) { ... }
```

- Comprobación

Se realizaron seis consultas consecutivas al saldo de Ana, obteniendo en todas ellas un saldo de $800.00 y el mismo valor de leidaEn. Aunque las respuestas fueron atendidas por tres réplicas diferentes del servicio cuenta.

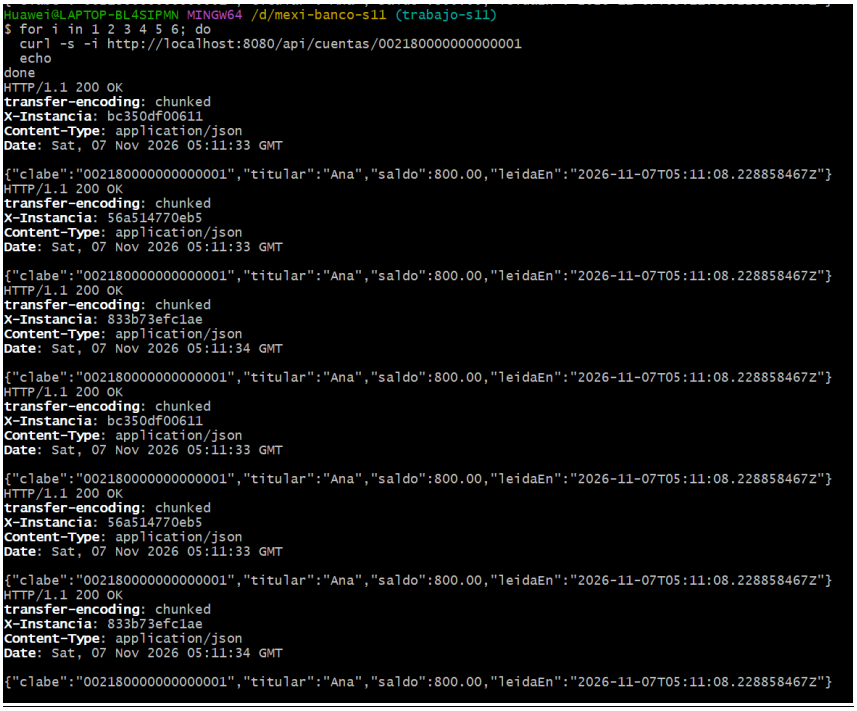


Figura. Seis consultas consecutivas del saldo de Ana atendidas por tres réplicas

Al revisar los registros del servicio cuenta, se encontró una sola lectura a PostgreSQL durante las seis consultas consecutivas. Esto demuestra que la primera petición almacenó el resultado en Redis y que las siguientes cinco utilizaron la información de la caché, evitando consultas innecesarias a la base de datos.

```
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s11 (trabajo-s11)
$ docker compose logs --since 2m cuenta | grep -c "leida de Postgres"
1
```

Figura. Redis almacena el saldo de Ana

Se verificó el contenido de Redis mediante el comando GET, obteniendo la información de la cuenta de Ana almacenada en formato JSON. Posteriormente, con el comando TTL se comprobó que la entrada tenía 15 segundos restantes antes de expirar. Esto demuestra que la caché utiliza el formato de serialización configurado y que las entradas tienen un tiempo de vida de 30 segundos.

```
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli GET cuentas::002180000000000001
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T05:22:14.249870677Z"}

Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli TTL cuentas::002180000000000001
15
```

Figura. GET y TTL

- Comprueba que la copia se borra

Se utilizó el comando MONITOR para observar en tiempo real las operaciones realizadas en Redis. Durante la transferencia se identificaron operaciones DEL sobre las claves de Ana y Beto, lo que demuestra que sus datos almacenados en caché fueron eliminados después de modificar sus saldos.

```
1794029343.398239 [0 127.0.0.1:42040] "ping"
1794029346.759191 [0 172.22.0.9:49626] "GET" "cuentas::002180000000000001"
1794029346.897687 [0 172.22.0.9:49626] "SET" "cuentas::002180000000000001" "{\"clabe\":\"002180000000000001\",\
}" "PX" "30000"
1794029346.904016 [0 172.22.0.14:35718] "GET" "cuentas::002180000000000002"
1794029346.908910 [0 172.22.0.14:35718] "SET" "cuentas::002180000000000002" "{\"clabe\":\"002180000000000002\",
\"}" "PX" "30000"
1794029347.203629 [0 172.22.0.8:52596] "DEL" "cuentas::002180000000000001"
1794029347.416921 [0 172.22.0.9:49626] "DEL" "cuentas::002180000000000002"
1794029348.430923 [0 127.0.0.1:36292] "ping"
```

Figura. Monitor de Redis

Se ejecutó el comando EXISTS para comprobar si la información de Ana continuaba almacenada en Redis. El resultado esperado fue 0, indicando que la clave ya no existía y que la copia anterior del saldo había sido eliminada correctamente.

```
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli EXISTS cuentas::002180000000000001
0
```

Figura. Verificación de existencia

Después de la transferencia, se volvió a consultar el saldo de Ana. Redis realizó una operación GET para buscar la información, pero al no encontrarla, obtuvo el nuevo saldo desde PostgreSQL y lo almacenó nuevamente mediante SET, con un tiempo de expiración de 30 segundos (PX 30000). El nuevo saldo registrado fue de $790.00.

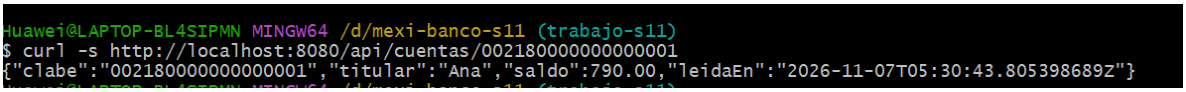


Figura. Consulta posterior a la transferencia

- Comporacion de cache

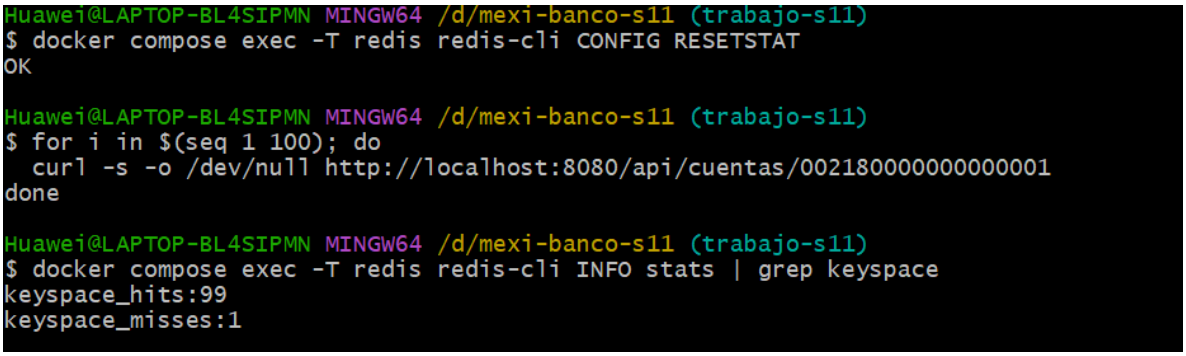


Figura. Prueba A

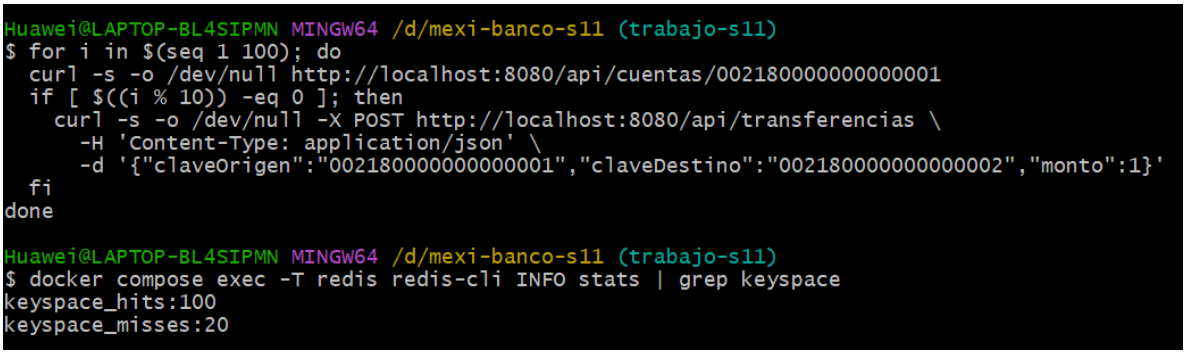


Figura. Prueba B

Comparación de escenarios

Escenario	Hits	Misses	Hit ratio
A: consultas seguidas	99	1	99 %
B: consultas y transferencias	100	20	83.33 %

En el escenario B, cada transferencia modifica los saldos de Ana y Beto, por lo que @CacheEvict elimina sus entradas de Redis. Cuando se vuelven a consultar, la información debe recuperarse de PostgreSQL y almacenarse nuevamente en la caché. Por eso aumentan los fallos y disminuye el hit ratio.

Se realizaron 100 consultas al saldo de Ana con la caché encendida y apagada. Con caché, el tiempo promedio fue de 13.10 ms, mientras que sin caché fue de 19.53 ms. Además, las lecturas a PostgreSQL disminuyeron de 100 a solamente 1 cuando Redis estaba habilitado. Esto demuestra que la caché mejora el tiempo de respuesta y reduce considerablemente las consultas directas a la base de datos.

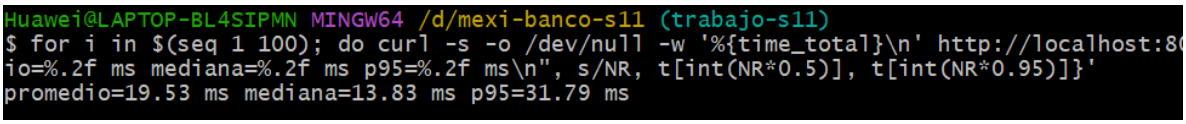


Figura. Sin cache

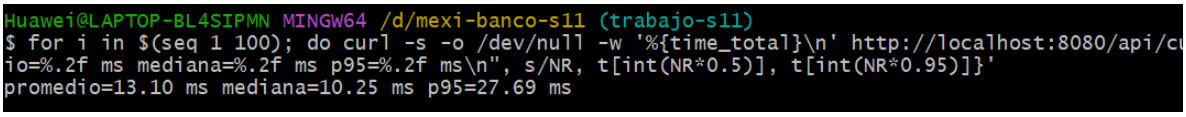


Figura. Con cache

Comparación final

Métrica	Con caché	Sin caché
Promedio	13.10 ms	19.53 ms
Mediana	10.25 ms	13.83 ms
P95	27.69 ms	31.79 ms
Lecturas a PostgreSQL	1	100

PARTE 2. Los avisos en MongoDB

- Comprobación de las tres formas de aviso

Se implementó el almacenamiento de avisos en MongoDB mediante la base de datos notificacion y la colección avisos. Cada aviso contiene campos comunes, como el identificador del evento, la CLABE, el tipo de transferencia, el monto, el saldo y la fecha. Además, se agregan campos específicos dependiendo del canal utilizado: SMS, CORREO o PUSH. De esta manera, cada documento conserva únicamente la información necesaria para su tipo de notificación.

Se realizaron dos transferencias: una de Ana a Beto y otra de Ana a Carla. Al consultar la colección avisos en MongoDB se obtuvieron cuatro registros: dos notificaciones SMS para Ana, un correo electrónico para Beto y una notificación PUSH para Carla. Esto permite comprobar que el sistema utiliza diferentes canales según el destinatario.

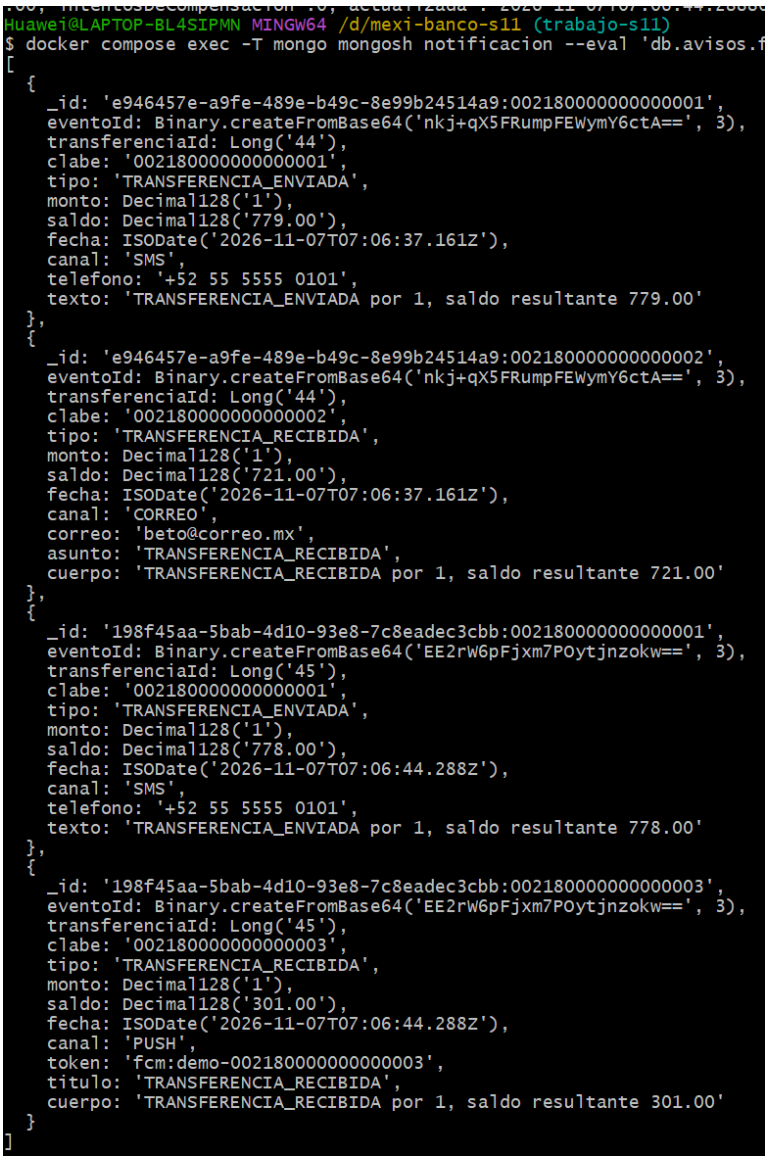


Figura. Colección de Avisos

Se realizó una consulta en MongoDB para mostrar únicamente los avisos que contienen el campo telefono, utilizando $exists. También se limitaron los campos mostrados a la CLABE, el canal y el teléfono. El resultado contiene solamente las dos notificaciones SMS correspondientes a Ana.

```
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.fin
[
  {
    _id: 'e946457e-a9fe-489e-b49c-8e99b24514a9:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: '198f45aa-5bab-4d10-93e8-7c8eadec3cbb:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  }
]
```

Figura. Consulta de avisos con teléfono

- Consulta de los últimos 10 avisos

Se realizaron doce transferencias de un peso de Ana a Beto para generar nuevos avisos. Posteriormente, se consultó la bitácora de Beto mediante el endpoint del gateway. El resultado mostró exactamente diez notificaciones por correo electrónico, ordenadas de la más reciente a la más antigua, comprobando que el servicio limita correctamente la cantidad de avisos devueltos.

$ curl -s http://localhost:8080/api/notificaciones/002180000000000002

[{"_id":"0ebda684-ef68-446f-9db3-3316e551cb81:002180000000000002","eventoId":"0ebda684-ef68-446f-9db3-3316e551cb81","transferenciaId":57,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":733.00,"fecha":"2026-11-07T07:13:42.892+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 733.00"},{"_id":"26b8ba45-babf-4a89-925a-0e7cdb647663:002180000000000002","eventoId":"26b8ba45-babf-4a89-925a-0e7cdb647663","transferenciaId":56,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":732.00,"fecha":"2026-11-07T07:13:42.687+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 732.00"},{"_id":"e6d24056-4bae-42a2-acdb-ebb5268d2c82:002180000000000002","eventoId":"e6d24056-4bae-42a2-acdb-ebb5268d2c82","transferenciaId":55,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":731.00,"fecha":"2026-11-07T07:13:42.486+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 731.00"},{"_id":"28019ddb-0a91-44f2-a42f-8902683e8a54:002180000000000002","eventoId":"28019ddb-0a91-44f2-a42f-8902683e8a54","transferenciaId":54,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":730.00,"fecha":"2026-11-07T07:13:42.296+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 730.00"},{"_id":"54eacbc5-344d-463d-81f3-2f0496279cb9:002180000000000002","eventoId":"54eacbc5-344d-463d-81f3-2f0496279cb9","transferenciaId":53,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":729.00,"fecha":"2026-11-07T07:13:42.097+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 729.00"},{"_id":"d052b5ab-f9db-4c3d-9291-ccd7ce03cbdf:002180000000000002","eventoId":"d052b5ab-f9db-4c3d-9291-ccd7ce03cbdf","transferenciaId":52,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":728.00,"fecha":"2026-11-07T07:13:41.896+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 728.00"},{"_id":"21141acb-b262-45ab-bc15-4866e774217a:002180000000000002","eventoId":"21141acb-b262-45ab-bc15-4866e774217a","transferenciaId":51,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":727.00,"fecha":"2026-11-07T07:13:41.710+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 727.00"},{"_id":"8ee9768f-bd49-4e10-bf67-f4692b2a58a9:002180000000000002","eventoId":"8ee9768f-bd49-4e10-bf67-f4692b2a58a9","transferenciaId":50,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":726.00,"fecha":"2026-11-07T07:13:41.518+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 726.00"},{"_id":"22152da5-2114-48dc-ab06-ae6301a08f8e:002180000000000002","eventoId":"22152da5-2114-48dc-ab06-ae6301a08f8e","transferenciaId":49,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":725.00,"fecha":"2026-11-07T07:13:41.314+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 725.00"},{"_id":"799c48c7-c7f3-4903-b672-f402c7c3a949:002180000000000002","eventoId":"799c48c7-c7f3-4903-b672-f402c7c3a949","transferenciaId":48,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":724.00,"fecha":"2026-11-07T07:13:41.114+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 724.00"}]



Figura. 10 avisos

- Lista de índices

Se consultaron los índices de la colección avisos mediante getIndexes(). Se comprobó la existencia del índice predeterminado _id_ y del índice compuesto clabe_fecha, que permite optimizar las consultas por CLABE y ordenar los avisos por fecha.

```
Huawei@LAPTOP-BL4SIPMN MINGW64 /d/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.getIndexes()'
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { clabe: 1, fecha: -1 }, name: 'clabe_fecha' }
]
```

Figura. Lista de índices

- Consulta utilizando el índice

Se utilizó explain("executionStats") para analizar la consulta de los últimos diez avisos de Beto. MongoDB utilizó el índice clabe_fecha, como se observa en la etapa IXSCAN, y examinó únicamente diez documentos para obtener los resultados solicitados.

```
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 10,
      inputStage: {
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { clabe: 1, fecha: -1 },
          indexName: 'clabe_fecha',
          isMultiKey: false,
          multiKeyPaths: { clabe: [], fecha: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'forward',
          indexBounds: {
            clabe: [ '["002180000000000002", "002180000000000002"]' ],
            fecha: [ '[MaxKey, MinKey]' ]
          }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 10,
    executionTimeMillis: 0,
    totalKeysExamined: 10,
    totalDocsExamined: 10,
    executionStages: {
```

Figura. Consultas con índice.

- Consulta utilizando sin índice

Se repitió para evitar el uso de índices. MongoDB realizó un recorrido completo mediante COLLSCAN y posteriormente ordenó los resultados mediante SORT. En este caso examinó 28 documentos para devolver los mismos diez avisos, mostrando la diferencia respecto a la consulta con índice.



Figura. Consultas sin índice.

Característica	Con índice	Sin índice
Etapas	LIMIT → FETCH → IXSCAN	SORT → COLLSCAN
Documentos examinados	10	28
Avisos devueltos	10	10
Índice utilizado	clabe_fecha	Ninguno

El índice clabe_fecha permitió obtener los diez avisos examinando únicamente diez documentos, mientras que sin índice MongoDB tuvo que recorrer los 28 documentos de la colección. Aunque la diferencia es pequeña en este conjunto de datos, el beneficio sería mucho mayor con colecciones de millones de registros.

PARTE 3. El vento duplicado

Se publicó dos veces el mismo evento desde la API de RabbitMQ, utilizando CLABEs de prueba para evitar afectar las cuentas existentes. En el primer procesamiento se generaron dos avisos, uno para el origen y otro para el destino. Al recibir nuevamente el mismo evento, el servicio detectó que ya había sido procesado y no generó documentos adicionales. Se comprobó en MongoDB que únicamente quedaron dos avisos, demostrando que el mecanismo de idempotencia evita duplicados.

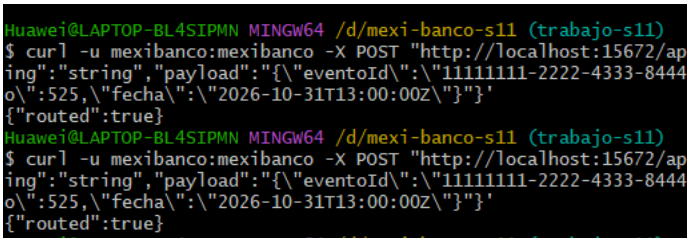


Figura. Publicación duplicada

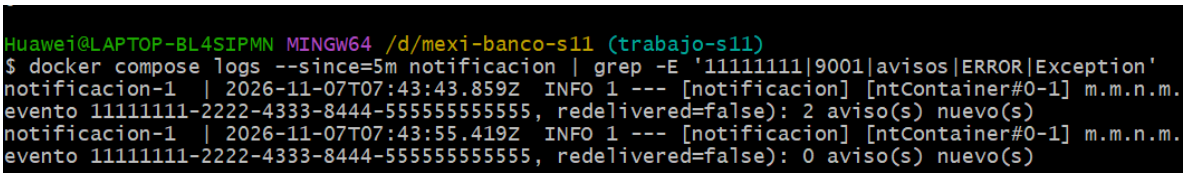


Figura . Logs

PARTE 4. El dato viejo

Se consultó el saldo de Ana para almacenarlo en Redis y posteriormente se modificó directamente en PostgreSQL a $5.00, sin utilizar la aplicación. Durante aproximadamente 32 segundos, el endpoint continuó mostrando el saldo anterior de $766.00. Cuando expiró el TTL de la caché, la aplicación consultó nuevamente PostgreSQL y mostró los $5.00 reales. También se intentó realizar una transferencia de $100, pero fue rechazada con el código HTTP 422 por saldo insuficiente. Finalmente, se restauró el saldo original de Ana a $766.00.



Figura . Cache desactualizada

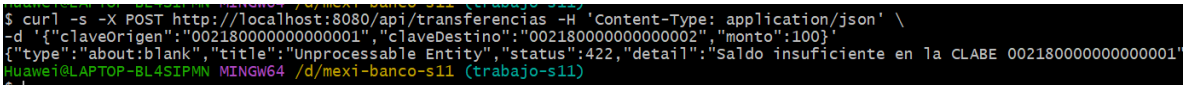


Figura. Transferencia rechazada

PREGUNTAS DE CIERRE

1. ¿Por qué MongoDB para los avisos y no para el saldo?

MongoDB se utiliza para los avisos porque permite guardar información diferente, como SMS, correos y notificaciones push. Para el saldo se utiliza PostgreSQL porque maneja mejor las transacciones y ayuda a evitar errores al realizar operaciones con dinero.

2. ¿Qué hit ratio midieron en A y en B, y por qué cambia? ¿Velocidad o capacidad?

En el escenario A se obtuvo un hit ratio de 99.0% y en el B de 84.2%. Disminuyó porque las transferencias eliminan los saldos de la caché para que la siguiente consulta obtenga información actualizada. La caché se utiliza principalmente para reducir las consultas que recibe PostgreSQL.

3. ¿Qué pasaría si la autorización del cargo leyera el saldo de la caché?

Podría autorizar transferencias con dinero que la cuenta ya no tiene, porque la caché puede conservar un saldo anterior durante algunos segundos. Esto provocaría errores en los saldos y permitiría gastar dinero inexistente.

4. Al aplicar un movimiento, ¿por qué DEL y no SET? ¿Y por qué después del commit?

Se utiliza DEL para eliminar el saldo guardado en Redis y hacer que la siguiente consulta obtenga el valor actualizado desde PostgreSQL. No se utiliza SET porque podría guardarse un saldo incorrecto si ocurren varias operaciones al mismo tiempo.

Se hace después del commit para asegurar que el cambio en PostgreSQL ya se haya guardado antes de eliminar la copia de Redis.